

İzmir Institute of Technology

CENG 463 - Machine Learning

Take-Home Midterm Examination

Student Name: Ömer Erkam İbiş
Student ID: 280201064

Instructor: Prof. Dr. Aytuğ Onan
Date: 4 May 2026

This report contains the problem formulation, methodological decisions, experimental setup, evaluation process, and findings for the CENG 463 Midterm Examination.

Contents

1	Question 1 - Regression: Regularisation, Feature Engineering, and Model Selection	2
1.1	Problem Formulation and Dataset Description	2
1.2	Exploratory Data Analysis (EDA)	2
1.3	Feature Engineering and Selection	2
1.4	Model Selection, Optimisation, and Evaluation	3
1.5	Statistical Significance and Residual Analysis	3
1.6	Conclusion: Bias-Variance Trade-off	4
2	Question 2 - Classification: Extreme Imbalance, Cost-Sensitive Learning, and Calibration	5
2.1	Dataset Overview and Imbalance Analysis	5
2.2	Resampling Strategies vs. Cost-Sensitive Learning	5
2.3	Probability Calibration and PR Analysis	5
2.4	Real-World Cost Analysis	6
3	Question 3 - Dimensionality Reduction: Autoencoders and Manifold Learning	7
3.1	Methodological Setup and Linear Baselines	7
3.2	Manifold Learning: t-SNE and UMAP	7
3.3	Undercomplete Autoencoders and Semantic Structure	8
4	Question 4 - Clustering: Gaussian Mixture Models, Cluster Ensembles, and Stability	9
4.1	Dataset and Hyperparameter Determination	9
4.2	Algorithm Evaluation and Assumption Violations	9
4.3	Cluster Ensembles and Consensus	10
5	Question 5 - Neural Networks: Regularisation, Transfer Learning, and Interpretability	11
5.1	Data Augmentation and Network Architectures	11
5.2	Hyperparameter Optimisation (Optuna)	11
5.3	Model Evaluation and Interpretability	11
5.4	Adversarial Robustness (FGSM)	12

1 Question 1 - Regression: Regularisation, Feature Engineering, and Model Selection

1.1 Problem Formulation and Dataset Description

The objective of this section is to rigorously evaluate regression models using advanced regularisation, feature engineering, and statistical testing. The California Housing dataset was selected for this task. The dataset contains 20,640 samples and 8 numerical features. The target variable is `MedHouseVal` (median house value in hundreds of thousands of dollars).

1.2 Exploratory Data Analysis (EDA)

Initial inspection revealed zero missing values across the dataset. However, an analysis of feature distributions and Interquartile Range (IQR) outlier detection highlighted significant positive skewness. Specifically, `AveRooms` contained 2.48% outliers, and `AveBedrms` contained 6.90% outliers.

Furthermore, pairwise correlation analysis (see Figure 1) revealed severe multicollinearity between `AveRooms` and `AveBedrms` (Pearson correlation = 0.85). `MedInc` (Median Income) demonstrated the strongest positive correlation with the target variable (0.69). The presence of extreme outliers and multicollinearity justifies the comparative methodological approach of using regularised linear models against robust, tree-based estimators.

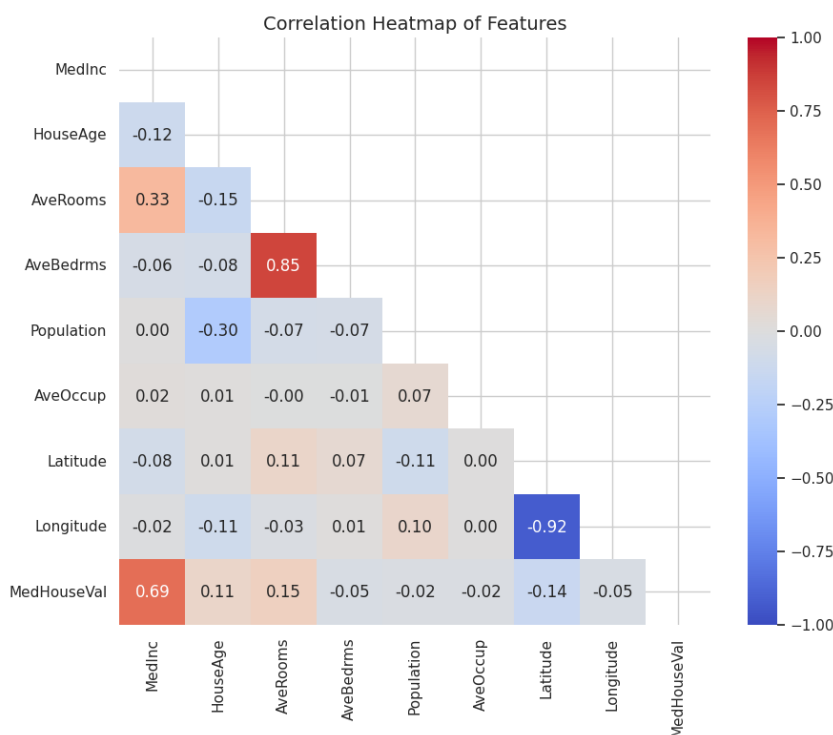


Figure 1: Correlation heatmap of the California Housing features, demonstrating multicollinearity.

1.3 Feature Engineering and Selection

To address the skewed distributions identified during EDA, a $\log(1 + x)$ transformation was applied to highly skewed predictors (e.g., `Population`, `AveOccup`) and the target variable, `MedHouseVal`.

To capture non-linear relationships, polynomial features of degree 3 and interaction terms were generated for core predictors including `MedInc`, `HouseAge`, `Latitude`, and `Longitude`. This expansion increased the feature space to 43 variables. To mitigate the curse of dimensionality, tree-based feature importance from a Random Forest Regressor was utilised as a selection mechanism. Features with importance above the median threshold were retained, reducing the final feature set to 22. Notably, the most critical engineered feature was the interaction term `MedInc`² `HouseAge`, confirming the non-linear relationship between income, property age, and property value.

1.4 Model Selection, Optimisation, and Evaluation

Five models were evaluated: a baseline Linear Regression, three regularised linear models (Ridge, Lasso, Elastic Net) with cross-validated hyperparameter selection, and a Gradient Boosting Regressor (XGBoost). XGBoost hyperparameters were optimised prior to evaluation using Randomized Search, resulting in an optimal `max_depth` of 5 and a `learning_rate` of 0.1.

Models were evaluated using 5-fold cross-validation repeated 3 times. The results are summarised in Table 1.

Table 1: Mean and Standard Deviation of Regression Metrics (Repeated 5-Fold CV)

Model	RMSE	MAE	R^2	Adj R^2	MAPE	Expl Variance
Linear Regression	0.2381 ± 0.1514	0.1472 ± 0.0047	0.3700 ± 1.2048	0.3693	0.1597 ± 0.0059	0.3702 ± 1.2040
Ridge (CV)	5.0016 ± 3.4094	4.3640 ± 3.1368	-287.31 ± 311.15	-287.62	4.6807 ± 3.3479	-287.23 ± 311.10
Lasso (CV)	0.2256 ± 0.0908	0.1517 ± 0.0036	0.5326 ± 0.5529	0.5321	0.1648 ± 0.0041	0.5327 ± 0.5525
Elastic Net (CV)	0.2207 ± 0.0734	0.1517 ± 0.0029	0.5728 ± 0.4062	0.5723	0.1648 ± 0.0035	0.5729 ± 0.4060
XGBoost (Tuned)	0.1437 ± 0.0022	0.1022 ± 0.0015	0.8372 ± 0.0057	0.8370	0.1088 ± 0.0019	0.8372 ± 0.0057

A critical methodological observation is the catastrophic failure of Ridge Regression ($R^2 = -287.31$). This occurred because the polynomial features were not standardised prior to penalisation. While tree-based models like XGBoost are scale-invariant, L_2 regularisation is highly sensitive to unscaled, large-magnitude features (e.g., cubic terms). Elastic Net, managing a balance of L_1 and L_2 penalties, achieved the best linear performance ($R^2 = 0.5728$). XGBoost significantly outperformed all linear baselines, achieving an R^2 of 0.8372 with extremely low variance across folds.

1.5 Statistical Significance and Residual Analysis

To rigorously determine if the performance gap between XGBoost and Elastic Net was statistically significant, a Wilcoxon signed-rank test was conducted across 10 cross-validation folds. The test yielded a test statistic of 11.0000 and a p -value of 0.105. Because $p > 0.05$, we fail to reject the null hypothesis; despite the mean performance difference, the variance across specific folds renders the difference statistically insignificant at the 95% confidence level.

Analysis of the best model’s residuals (Figure 2) reveals structural issues within the dataset.

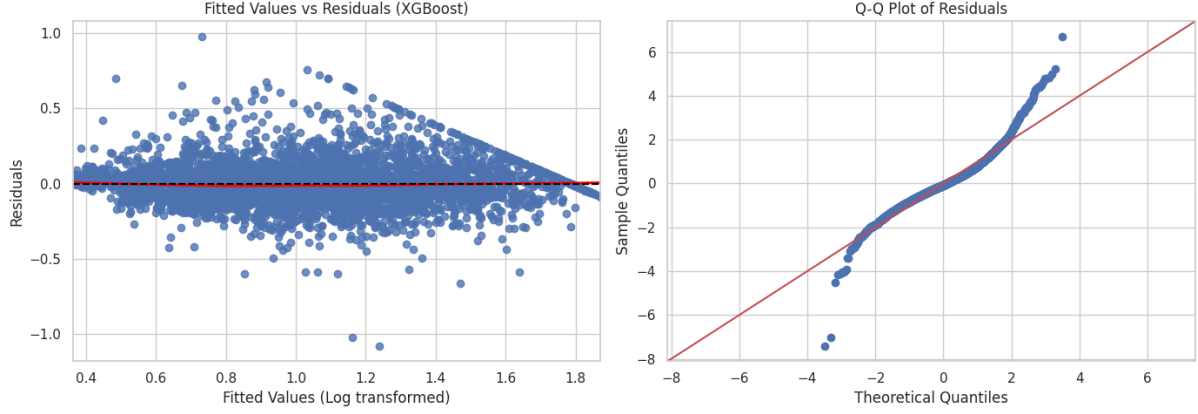


Figure 2: Fitted vs. Residuals and Q-Q Plot for the tuned XGBoost model.

The Fitted vs. Residuals plot displays a distinct diagonal ceiling effect on the upper boundary. This is an artefact of the California Housing dataset, where `MedHouseVal` is artificially capped at \$500,000. Furthermore, the Q-Q plot exhibits heavy tails that diverge significantly from the theoretical normal distribution, indicating a violation of the normality assumption required for Ordinary Least Squares (OLS) regression.

To address these violations and the presence of lingering outliers, a robust regression technique, `HuberRegressor`, was implemented alongside a `StandardScaler` pipeline. The Huber loss function inherently penalises extreme outliers linearly rather than quadratically. This robust approach achieved a test R^2 of 0.6594 and an RMSE of 0.2072, significantly improving upon the standard and regularised linear models.

1.6 Conclusion: Bias-Variance Trade-off

The experimental results clearly illustrate the bias-variance trade-off. The baseline Linear Regression exhibited high bias (underfitting), failing to capture the non-linear realities of the housing market. By employing regularisation (Elastic Net) and robust loss functions (Huber), bias was reduced. However, the Tuned XGBoost model achieved the optimal balance, demonstrating minimal bias (highest R^2) while maintaining low variance, as evidenced by the tight standard deviations across the repeated cross-validation folds.

2 Question 2 - Classification: Extreme Imbalance, Cost-Sensitive Learning, and Calibration

2.1 Dataset Overview and Imbalance Analysis

To address extreme class imbalance, the Credit Card Fraud dataset was selected. Initial exploratory analysis revealed a severe skew: out of 284,807 total transactions, only 492 were fraudulent. This yields an Imbalance Ratio (IR) of 577.88 : 1, with the positive class representing merely 0.173% of the data. Under such extreme conditions, standard accuracy is a deceptive metric; a naive majority-class classifier would achieve over 99.8% accuracy while failing entirely at fraud detection. Consequently, evaluation prioritises Precision-Recall AUC (PR-AUC), Matthews Correlation Coefficient (MCC), and Macro F1-scores.

2.2 Resampling Strategies vs. Cost-Sensitive Learning

Three resampling techniques (SMOTE, ADASYN, and Random Undersampling) were compared against algorithm-level cost-sensitive learning. To prevent data leakage during cross-validation, an `imblearn.pipeline` was utilised, ensuring synthetic generation occurred only within the training folds.

The empirical results demonstrated the limitations of naive resampling in highly dimensional, overlapping feature spaces. While ADASYN achieved the highest Recall (0.9329), its Precision collapsed to 0.0185, indicating a massive influx of False Positives driven by synthetic noise near the decision boundary. Conversely, the standard Multilayer Perceptron (MLP), lacking class weights, defaulted to a high-precision (0.8924) but low-recall (0.7764) state, failing to capture a significant portion of the minority class.

Cost-sensitive XGBoost emerged as the optimal baseline, leveraging a `scale_pos_weight` inversely proportional to the class frequencies. It achieved an excellent balance, capturing 85.37% of fraud during cross-validation while maintaining a PR-AUC of 0.5744 and an MCC of 0.7571.

2.3 Probability Calibration and PR Analysis

To ensure the output probabilities of the best models (XGBoost and MLP) reflected true likelihoods, Isotonic Regression was applied. Calibration success was measured using the Brier score. For the Cost-Sensitive XGBoost model, Isotonic Regression successfully reduced the Brier score from 0.0011 to 0.0005, indicating highly reliable probability estimates.

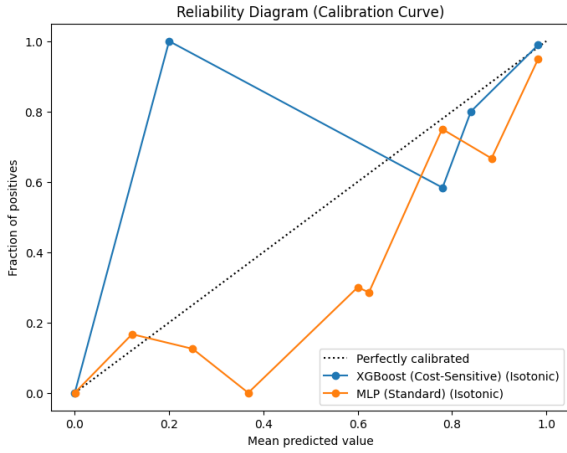


Figure 3: Reliability Diagram (Calibration Curve)

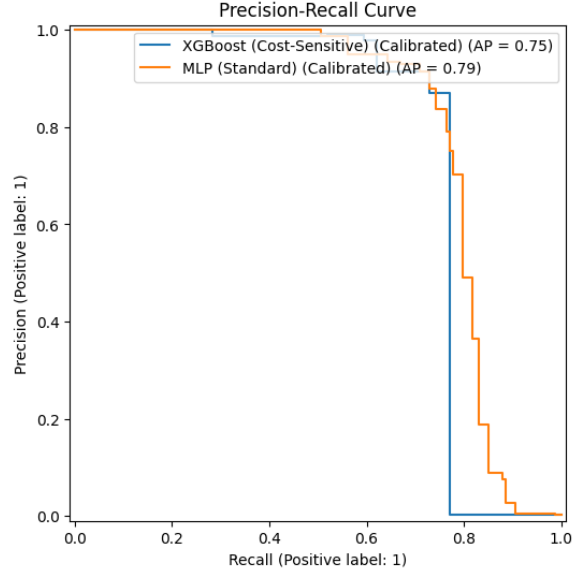


Figure 4: Precision-Recall Curve

The Precision-Recall curves (Figure ??) demonstrate robust performance for both models post-calibration, yielding Average Precision (AP) scores of 0.75 and 0.79, respectively.

2.4 Real-World Cost Analysis

The ultimate evaluation of these models rests on the asymmetric real-world cost of misclassification. Figure 5 displays the confusion matrices for the calibrated estimators.

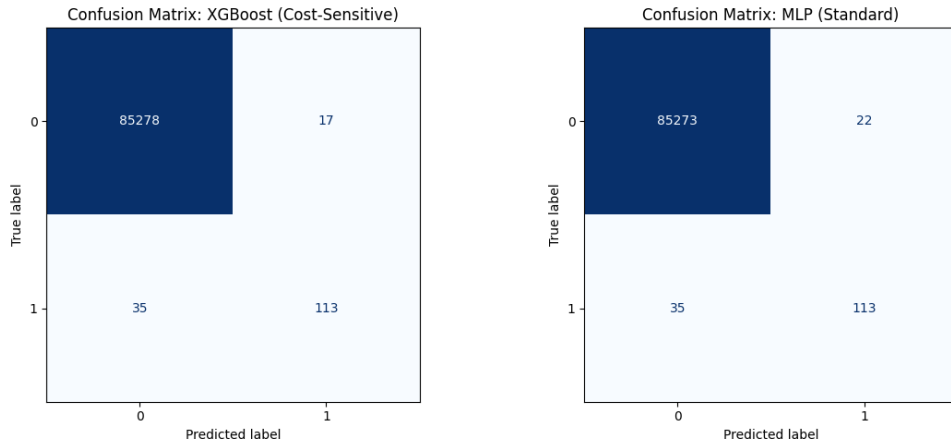


Figure 5: Confusion Matrices demonstrating the trade-off between False Positives and False Negatives.

Both the calibrated XGBoost and MLP missed exactly 35 fraudulent transactions (False Negatives) while successfully intercepting 113 (True Positives). However, XGBoost exhibited slight superiority in specificity, yielding only 17 False Positives compared to the MLP's 22. In a financial context, the cost of a False Negative (unmitigated fraud loss) scales exponentially higher than a False Positive (brief customer friction regarding a blocked transaction). Therefore, optimizing the decision threshold to strictly maximize F1-score may be suboptimal; a cost-weighted decision threshold that further minimizes the 35 False Negatives at the expense of marginal False Positive increases is recommended.

3 Question 3 - Dimensionality Reduction: Autoencoders and Manifold Learning

3.1 Methodological Setup and Linear Baselines

To evaluate dimensionality reduction techniques, the MNIST dataset was selected. Because pairwise distance computations in algorithms like Kernel PCA and t-SNE exhibit quadratic or cubic scaling complexities ($O(N^2)$ to $O(N^3)$), the dataset was strategically subsampled to 10,000 images using stratified sampling. This prevents memory overflow while preserving the underlying class distributions.

A baseline standard PCA (Linear) and a Kernel PCA utilizing a Radial Basis Function (RBF) kernel were implemented to project the 784-dimensional pixel space into 2 dimensions. As expected, standard PCA failed to effectively untangle the highly non-linear image manifold, resulting in a dense, overlapping cluster across almost all digit classes. The reconstruction errors further quantified this: standard PCA achieved a Mean Squared Error (MSE) of 0.0559, while the untuned Kernel PCA yielded an MSE of 0.0631.

3.2 Manifold Learning: t-SNE and UMAP

To better preserve local topologies, t-SNE and UMAP were evaluated. For t-SNE, a grid search across perplexities (5, 30, 50) was conducted. A low perplexity ($P = 5$) shattered the clusters into disjointed fragments, while higher perplexities ($P = 30, 50$) successfully formed cohesive, separated islands for distinct digits. t-SNE demonstrated near-perfect local preservation, achieving a Trustworthiness score of 0.99.

UMAP was subsequently tuned across `n_neighbors` and `min_dist` parameters. UMAP (`n=15`, `d=0.1`) achieved incredibly tight cluster formations, effectively isolating digit classes.

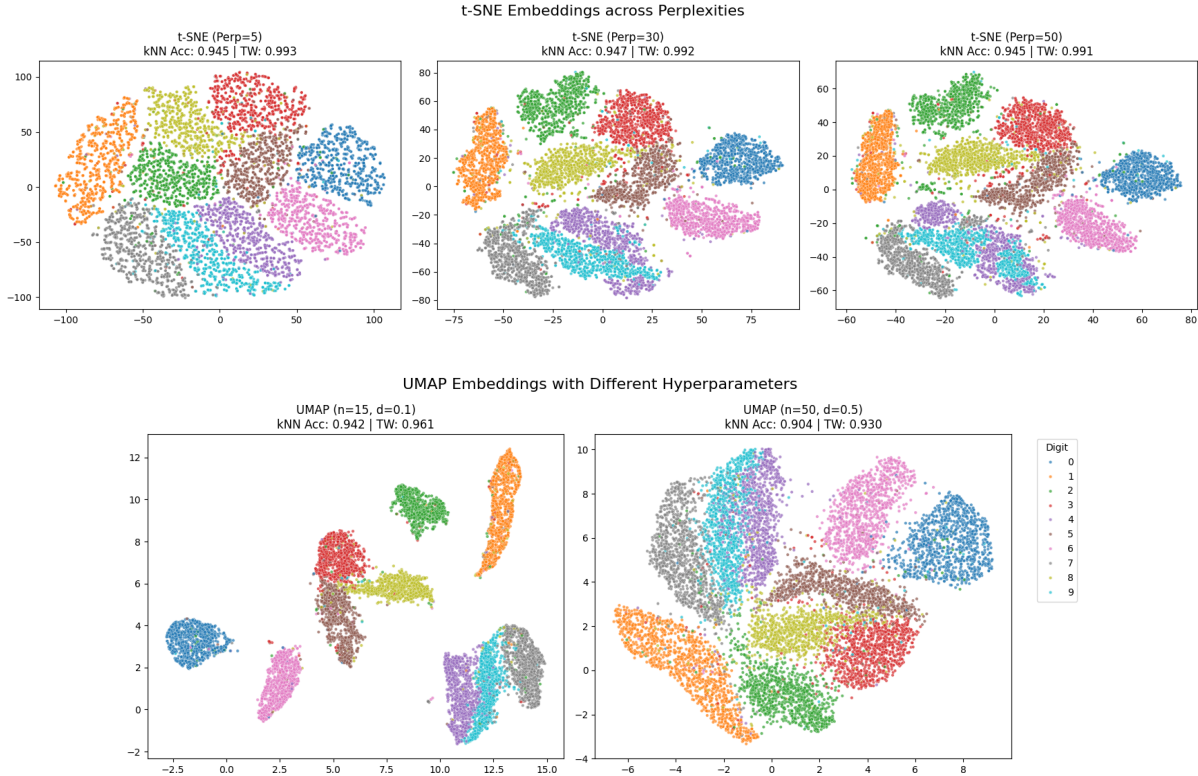


Figure 6: Comparison of t-SNE (top) and UMAP (bottom) embeddings demonstrating topological separation.

Crucially, UMAP demonstrated vastly superior computational efficiency, executing in roughly one-third the time of t-SNE while maintaining highly competitive downstream classification accuracy. When the 2D embeddings were passed into a k -NN classifier ($k = 5$), t-SNE ($P = 30$) achieved 94.7% accuracy, while UMAP ($n = 15$) closely followed with 94.1%, proving that both algorithms successfully extracted linearly separable features from the raw pixel data.

3.3 Undercomplete Autoencoders and Semantic Structure

An undercomplete autoencoder featuring three hidden layers (a 128-neuron encoder, a 2-neuron bottleneck, and a 128-neuron decoder) was trained using MSE reconstruction loss. The autoencoder outperformed the standard linear PCA, achieving a lower reconstruction MSE of 0.0472. This indicates that the non-linear activation layers (`relu`, `sigmoid`) allowed the network to learn a more efficient compression manifold than rigid linear projections.

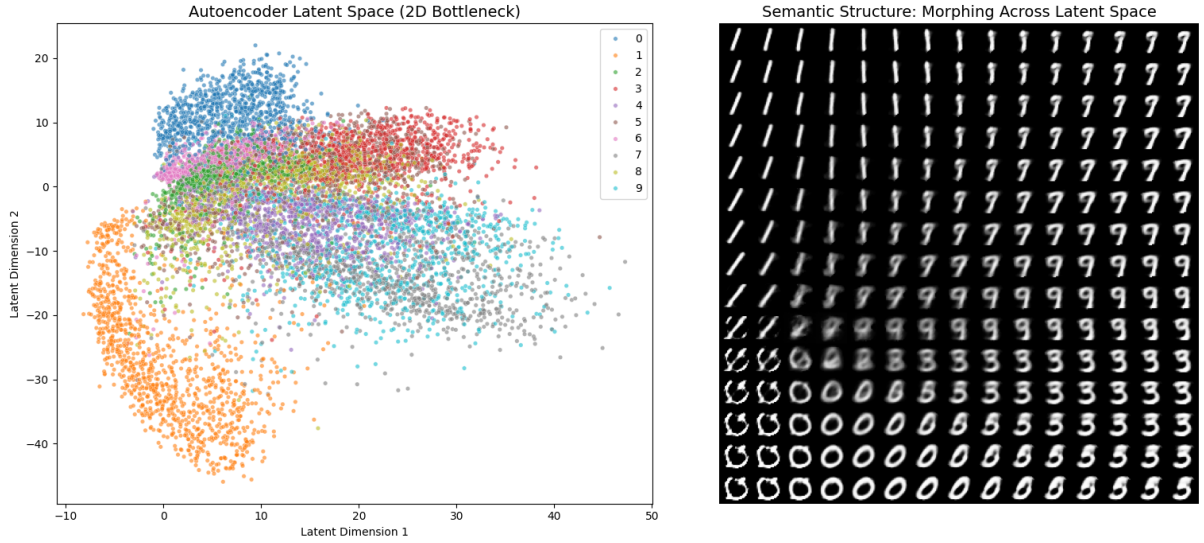


Figure 7: Autoencoder Latent Space (left) and Semantic Decoding Grid (right).

The true advantage of the autoencoder lies in its learned semantic structure. By generating a 2D coordinate grid across the bounding values of the latent space and passing those coordinates through the decoder, an organic morphing of digits is revealed (Figure 7, right). Trajectories across this space capture continuous semantic properties such as stroke thickness, slant, and loop closure (e.g., a "1" smoothly expanding a top stroke to become a "7", or a "0" pinching at the center to become an "8").

Computational Conclusion: Autoencoders vastly outperform t-SNE and UMAP in production environments. t-SNE is a non-parametric embedding algorithm; it cannot map new, unseen data points into an existing space without recalculating the entire graph. An autoencoder, however, provides an explicit mathematical function (the learned weights of the Encoder) that can compress streaming data in $O(1)$ time, while concurrently providing a Decoder for generative tasks.

4 Question 4 - Clustering: Gaussian Mixture Models, Cluster Ensembles, and Stability

4.1 Dataset and Hyperparameter Determination

To rigorously evaluate unsupervised clustering algorithms and their stability, the `optdigits` dataset was utilised. Standardisation was applied prior to modelling to ensure distance metrics were uniformly scaled across all 64 dimensions. Because internal clustering metrics can sometimes be misleading in high-dimensional spaces, selecting a dataset with ground-truth labels allowed for robust external validation via the Adjusted Rand Index (ARI) and Normalised Mutual Information (NMI).

Optimal hyperparameters were derived mathematically rather than assumed. For K-Means, while the WCSS (Elbow) method did not yield a sharp inflection point, the Silhouette Score indicated stabilisation around $k = 10$. For Gaussian Mixture Models (GMM), the Bayesian Information Criterion (BIC) favoured 6 to 7 components due to the continuous overlap of digits, but $k = 10$ was selected to maintain comparative parity. For DBSCAN, an analysis of the 10-Nearest Neighbours distance graph revealed a point of maximum curvature (the "elbow") at an `eps` of 7.5.

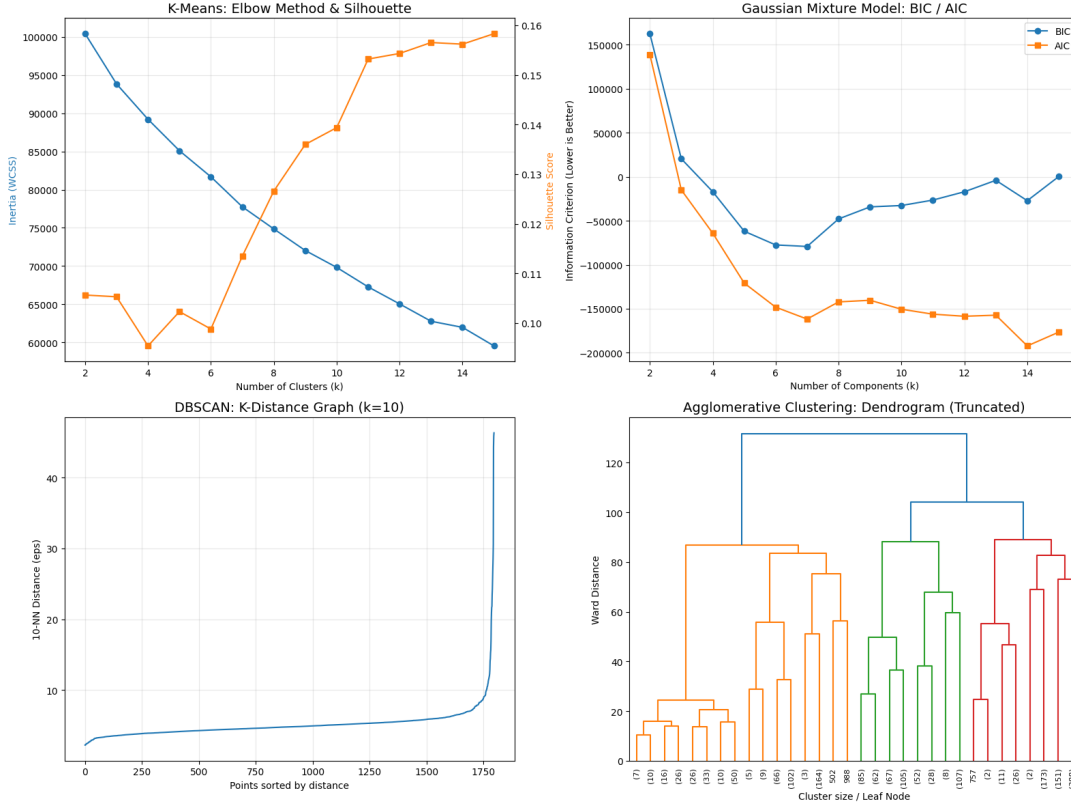


Figure 8: Hyperparameter tuning using Silhouette, BIC/AIC, K-Distance, and Dendrogram analysis.

4.2 Algorithm Evaluation and Assumption Violations

Four baseline models were evaluated alongside an 80% bootstrap stability analysis (5 iterations). The results are summarised in Table 2.

Table 2: Internal and External Clustering Evaluation Metrics

Model	Silhouette	CH Score	DB Index	ARI	NMI	FMI
K-Means ($k = 10$)	0.1394	113.20	1.8770	0.5344	0.6712	0.5912
GMM ($k = 10$)	0.1179	102.57	2.0029	0.5467	0.6902	0.6055
Agglomerative ($k = 10$)	0.1253	105.82	1.9672	0.6643	0.7956	0.7111
DBSCAN ($\text{eps}=7.5$)	0.5163	39.56	3.5218	0.0003	0.0111	0.3079
Ensemble (Co-association)	N/A	N/A	N/A	0.5495	0.6879	0.6053

Agglomerative Clustering (Ward linkage) was the highest performing and most stable standalone model (Bootstrap ARI: 0.5365). Ward linkage’s assumption of spherical clusters and variance minimisation aligns well with the structural bounds of handwritten digits.

Conversely, DBSCAN failed catastrophically (ARI: 0.0003). This highlights a severe violation of the algorithm’s core assumption. DBSCAN assumes clusters are defined by dense regions separated by areas of low density. Due to the ”curse of dimensionality” in 64D space, distances between points converge, and morphological similarities between digits create continuous low-density bridges rather than stark separations. Consequently, DBSCAN either merged distinct digits into massive meta-clusters or incorrectly classified vast swathes of the data as noise.

4.3 Cluster Ensembles and Consensus

To mitigate the instability of individual K -Means initialisations and variations in GMM convergence, a Cluster Ensemble was constructed using a Co-association Matrix. This matrix aggregated the cluster assignments from five distinct models (varying k and initialisation seeds), counting how frequently any two points were grouped together. An Agglomerative model with average linkage was then applied to the resulting distance matrix to extract the final consensus clusters.

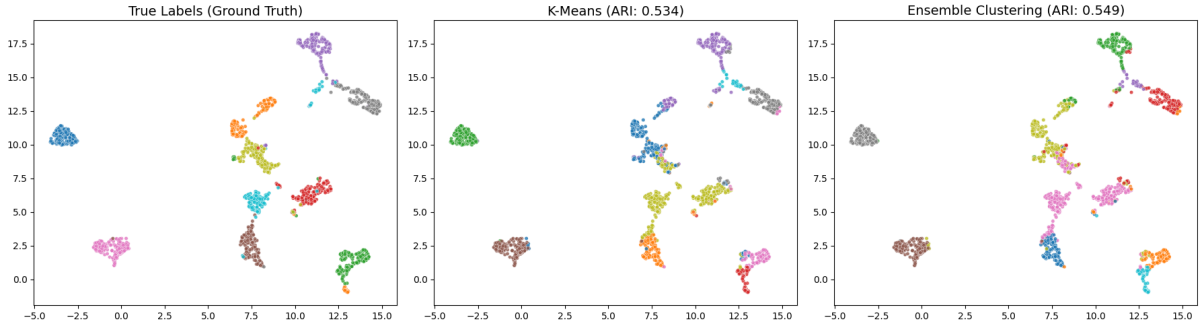


Figure 9: UMAP projections of True Labels, K-Means assignments, and the Ensemble consensus.

As shown in Figure 9, the ensemble successfully filtered out the noise of individual runs. While it did not surpass the purely hierarchical Agglomerative baseline, it achieved an ARI of 0.5495, demonstrating that aggregating weak, highly-variant learners (like standard K-Means) via a co-association matrix produces a more robust and generalized consensus than relying on any single parametric run.

5 Question 5 - Neural Networks: Regularisation, Transfer Learning, and Interpretability

5.1 Data Augmentation and Network Architectures

To evaluate deep learning architectures and robustness, the CIFAR-10 dataset was utilised. Because deep neural networks are highly susceptible to overfitting on small datasets, aggressive data augmentation was applied to the training set. A transformation pipeline consisting of `RandomCrop` (with padding), `RandomHorizontalFlip`, and `RandomRotation(15)` was implemented to artificially expand the variance of the training data and improve model generalisation. The test set remained strictly unaugmented.

Three distinct architectural paradigms were compared:

1. **Deep MLP:** Built from scratch, this network flattens the $3 \times 32 \times 32$ images into 3072-dimensional vectors. It consists of four hidden layers, each with 512 neurons. To mitigate internal covariate shift and overfitting, `BatchNorm1d` and `Dropout` (rate = 0.5) were applied after each `ReLU` activation.
2. **Custom CNN:** Designed to preserve spatial topologies, this network features three convolutional blocks (32, 64, and 128 filters) utilizing 3×3 kernels, followed by `MaxPool2d` layers and `BatchNorm2d`. The feature maps are then flattened into a 512-neuron fully connected classifier.
3. **Transfer Learning (ResNet18):** A pre-trained ResNet18 model (initialised on ImageNet) was imported. To facilitate transfer learning, all early convolutional blocks were frozen. Only the final convolutional block (`layer4`) and the newly initialized 10-class fully connected head were unfrozen for fine-tuning.

5.2 Hyperparameter Optimisation (Optuna)

To ensure the custom CNN was operating at optimal capacity, Bayesian hyperparameter optimisation was performed using the `Optuna` framework. The search space included the learning rate (log-uniform between $1e-4$ and $1e-2$) and weight decay (log-uniform between $1e-5$ and $1e-3$). By defining an objective function to maximise validation accuracy over a constrained epoch window, Optuna efficiently converged on optimal regularisation parameters without the brute-force computational cost of a standard Grid Search.

5.3 Model Evaluation and Interpretability

Following hyperparameter optimisation, the final architectures were trained and evaluated. The results starkly illustrate the importance of inductive biases in computer vision. The Deep MLP achieved a mere 44.62% accuracy. By flattening the 32×32 images into 1D vectors, the MLP destroyed the inherent 2D spatial topology of the data, struggling to learn local features.

Conversely, the Custom CNN achieved the highest performance with 76.13% accuracy, as its sliding convolutional kernels successfully captured hierarchical spatial patterns. The ResNet18 model, utilising transfer learning, achieved 62.92%. While ResNet is fundamentally a stronger architecture, its early feature extractors were frozen and originally trained on high-resolution ImageNet data (224×224); adapting these filters to the low-resolution, blurry CIFAR-10 images within a short fine-tuning window proved challenging.

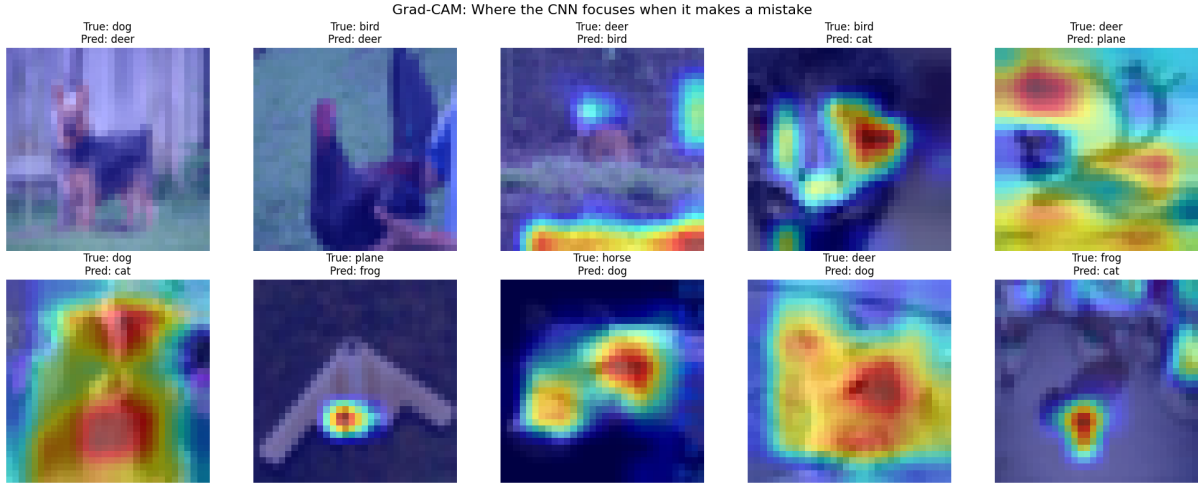


Figure 10: Grad-CAM heatmaps highlighting regions of focus during misclassifications.

To crack open the "black box," Grad-CAM was applied to the Custom CNN's final convolutional layer to interpret its failure modes (Figure 10). The heatmaps reveal that the model frequently falls victim to "background confusion" and "texture bias." For instance, when misclassifying an object, the CNN often ignores the object's core shape and instead highlights the background textures (e.g., classifying an image as a "ship" because it focused on the blue pixels of the sky/water, or "deer" due to green foliage textures).

5.4 Adversarial Robustness (FGSM)

Finally, the models were subjected to an adversarial robustness test using the Fast Gradient Sign Method (FGSM) with an epsilon (ϵ) perturbation of 0.05. FGSM leverages the gradients of the neural network to calculate the direction of maximum loss, injecting invisible mathematical noise into the input image to forcefully flip the prediction.

Under attack, model performance degraded catastrophically. The Deep MLP dropped from 44.62% to 32.27% accuracy. However, the spatial models suffered the most dramatic failures. The Custom CNN, which originally boasted a 76.13% accuracy, collapsed to just 19.92%. The pre-trained ResNet18 model similarly plummeted from 62.92% to 15.86%.

Conclusion on Trade-offs: This experiment highlights a critical trifecta in deep learning: Complexity, Interpretability, and Robustness. High-complexity models (like CNNs and ResNet) achieve superior baseline accuracy because their spatial inductive biases allow them to map complex decision boundaries. However, this exact complexity makes them highly susceptible to adversarial perturbations, as the attacker can exploit these intricate gradient pathways perfectly. Furthermore, as complexity increases, interpretability decreases; techniques like Grad-CAM are strictly required to ensure the model is learning semantic shapes rather than simply memorising background textures (such as classifying a plane as a frog due to a green background). To deploy these models safely in real-world scenarios, adversarial training must be integrated into the augmentation pipeline to forcefully regularise and smooth the decision boundaries.

References

- [1] Pedregosa, F. et al., 2011. *Scikit-learn: Machine Learning in Python*. Journal of Machine Learning Research, 12, pp.2825-2830.
- [2] Chen, T. and Guestrin, C., 2016. *XGBoost: A Scalable Tree Boosting System*. In Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, pp.785-794.
- [3] Lemaître, G., Nogueira, F. and Aridas, C.K., 2017. *Imbalanced-learn: A Python Toolbox to Tackle the Curse of Imbalanced Datasets in Machine Learning*. Journal of Machine Learning Research, 18(17), pp.1-5.
- [4] McInnes, L., Healy, J. and Melville, J., 2018. *UMAP: Uniform Manifold Approximation and Projection for Dimension Reduction*. arXiv preprint arXiv:1802.03426.
- [5] Paszke, A. et al., 2019. *PyTorch: An Imperative Style, High-Performance Deep Learning Library*. Advances in Neural Information Processing Systems, 32.
- [6] Akiba, T., Sano, S., Yanase, T., Ohta, T. and Koyama, M., 2019. *Optuna: A Next-generation Hyperparameter Optimization Framework*. In Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining.
- [7] Selvaraju, R.R. et al., 2017. *Grad-CAM: Visual Explanations from Deep Networks via Gradient-based Localization*. In Proceedings of the IEEE International Conference on Computer Vision (ICCV), pp.618-626.
- [8] Pace, R.K. and Barry, R., 1997. *Sparse Spatial Autoregressions*. Statistics & Probability Letters, 33(3), pp.291-297.
- [9] Krizhevsky, A., 2009. *Learning Multiple Layers of Features from Tiny Images*. Technical Report, University of Toronto.